Introduction to Programming - Fall 2026

Loops 🔁

A ~~picture~~ **loop** is worth a thousand ~~words~~ **lines**.

Agenda

01	Recap & AMA
02	Why we need loops
03	`for` loops with `range()`
04	`while` loops
05	`break` and `continue`
–	Break (15 min)
06	Common loop patterns – lists, strings & nested loops
07	Homework

Parts 03, 04, 05 and 06 have short exercises. ✍️

01 - Recap from last week

Last week we covered:

- ✂️ **Strings** – `+`, `*`, `len()`, and methods like `.lower()` and `.strip()`
- 🧩 **f-strings** – `f"Hello, {name}!"` to build clean output
- ⚖️ **Comparisons** – `==`, `!=`, `<`, `>`, `<=`, `>=`, and `in`
- 🔗 **Logical operators** – `and`, `or`, `not`
- 🔀 **Decisions** – `if` / `elif` / `else`, nesting, and truthy & falsy values

Today we teach our programs to **repeat** things.

Quick Reminder: Strings

- `+` joins strings, `*` repeats them
- `len()` counts characters, including spaces
- **Methods** are actions you call with a dot `.`
- Methods return a **new** string. Store it to keep it.
- f-strings put values inside text with `{ }`
- `in` checks if a string **contains** some text

```
name = "  alice  "
clean = name.strip().capitalize()

print(clean)              # Alice
print(clean.upper())      # ALICE
print(len(clean))         # 5
print("ha" * 3)           # hahaha
print(f"Hi, {clean}!")    # Hi, Alice!
print("lic" in clean)     # True
print("bob" in clean)     # False

answer = input("yes or no? ").strip().lower()
```

Quick Reminder: `if` / `elif` / `else`

Python checks each condition **from top to bottom**.

- Only the **first** true condition runs.
- `else` catches everything else.
- Two outcomes? `if` / `else` is enough.
- More than two? Add `elif`.
- Join checks with `and`, `or`, `not`.

```
temperature = int(input("Temperature? "))
weather = input("Weather? ").strip().lower()

if weather == "rain" or weather == "snow":
    print("Take an umbrella.")
elif temperature > 25 and weather == "sunny":
    print("Wear a T-shirt.")
elif temperature > 15:
    print("Bring a jacket.")
else:
    print("Wear a coat.")
```

Questions?

02 - What Is a Loop?

A **loop** is a block of code that runs **again and again**: a fixed number of times, or until something tells it to stop.

Why do we need loops?

- Print 100 lines without writing 100 prints
- Go through every item in a list
- Keep asking until the user types `"quit"`
- Read every line of a file

```
# Without a loop:
print("Hello!")
print("Hello!")
print("Hello!")
print("Hello!")
print("Hello!")

# With a loop:
for i in range(5):
    print("Hello!")
```

03 - Anatomy of a `for` Loop

```
for i in range(5):
    print(f"Round {i}")
```

- `for` … `in` – keywords that shape the loop
- `i` – the **loop variable**. You pick the name. Each iteration, it holds the next number, and the body can use it
- `range(5)` – the sequence we loop over: the numbers `0, 1, 2, 3, 4`
- `:` + indent – the end of the first line. The indented code is the **body**, which runs once per item

Step by Step: What `i` Holds

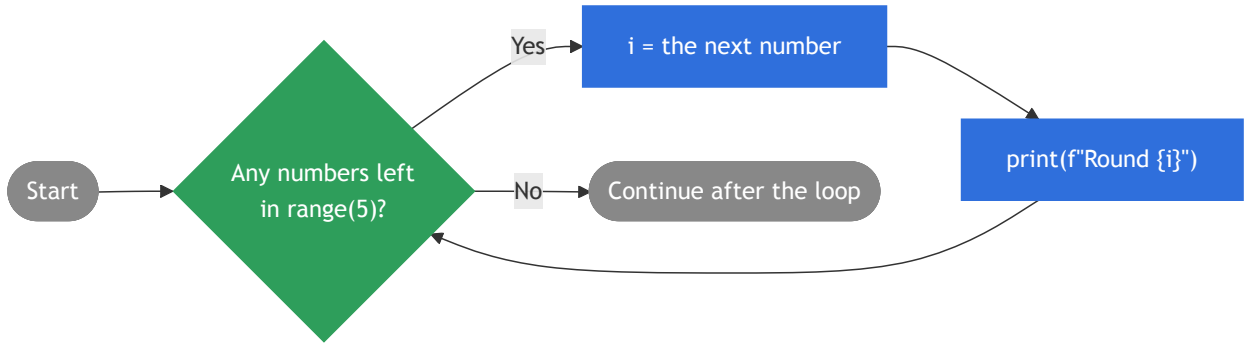
```
for i in range(5):
    print(f"Round {i}")
```

Each time the body runs is called an **iteration**.
Python gives `i` a new value before every iteration.

After the last number, there's nothing left, so the loop ends.

Iteration	`i` is	It prints
1st	`0`	`Round 0`
2nd	`1`	`Round 1`
3rd	`2`	`Round 2`
4th	`3`	`Round 3`
5th	`4`	`Round 4`

How a `for` Loop Runs



In Plain English

for each **element** **in** this **range / collection**:

    do something

Inside or After the Loop?

Indented – inside the loop, repeats

```
for i in range(3):
    print(f"Round {i}")
    print("Done!")
```

```
Round 0
Done!
Round 1
Done!
Round 2
Done!
```

Not indented – after the loop, runs once

```
for i in range(3):
    print(f"Round {i}")
print("Done!")
```

```
Round 0
Round 1
Round 2
Done!
```

Only the **indented** lines belong to the loop. The first line that isn't indented runs once, after the loop ends.

WEEK 5 – LOOPS

Exercise 1 – Lap Counter · 5 min

✍️ A runner does 4 laps:

1. Use a `for` loop to print one line per lap
2. After the loop, print `Race finished!` once

Need a hint?

- `{i + 1}` makes laps start at 1

Syntax reminder:

```
for i in range(count):
    # runs every pass
# runs once, after the loop
```

Expected output:

```
Lap 1 done
Lap 2 done
Lap 3 done
Lap 4 done
Race finished!
```

WEEK 5 – LOOPS

Exercise 1 – Advanced

🔵 Finished early? Now the runner does `laps = 5` laps:

1. Print one line per lap, and mark the **last** lap – when `i + 1` equals `laps`
2. After the loop, print `Race finished!` once

Syntax reminder:

```
if condition:
    # the special case
else:
    # every other pass
```

Expected output:

```
Lap 1 done
Lap 2 done
Lap 3 done
Lap 4 done
Lap 5 done - last lap!
Race finished!
```

`range()`: Start, Stop, Step

One argument: stop

```
for i in range(5):
    print(i)
```

Prints `0, 1, 2, 3, 4`

Two arguments: start, stop

```
for i in range(1, 6):
    print(i)
```

Prints `1, 2, 3, 4, 5`

Three arguments: start, stop, step

```
for i in range(0, 10, 2):
    print(i)
```

Prints `0, 2, 4, 6, 8`

Negative step: count down 🚀

```
for i in range(5, 0, -1):
    print(i)
```

Prints `5, 4, 3, 2, 1`

`range` always stops **before** the stop number. That's why `range(5)` ends at `4`, and `range(5, 0, -1)` ends at `1`.

Exercise 2 – Even Numbers with `for` · 5 min

✍️ Print every **even** number from **0 to 20**, including 20, one per line.

Use a `for` loop.

Syntax reminder:

```
for number in range(start, stop, step):
    print(number)
# stop itself is never included
```

Expected output:

```
0
2
4
6
8
10
12
14
16
18
20
```

Exercise 2 – Advanced

🔵 Finished early? A leap year comes every 4 years.

1. Print every leap year from **2000 to 2040**, including 2040
2. After the loop, print the **next** leap year

Syntax reminder:

```
for year in range(start, stop, step):
    print(f"{year} is a leap year")
# year keeps its last value here
```

Expected output:

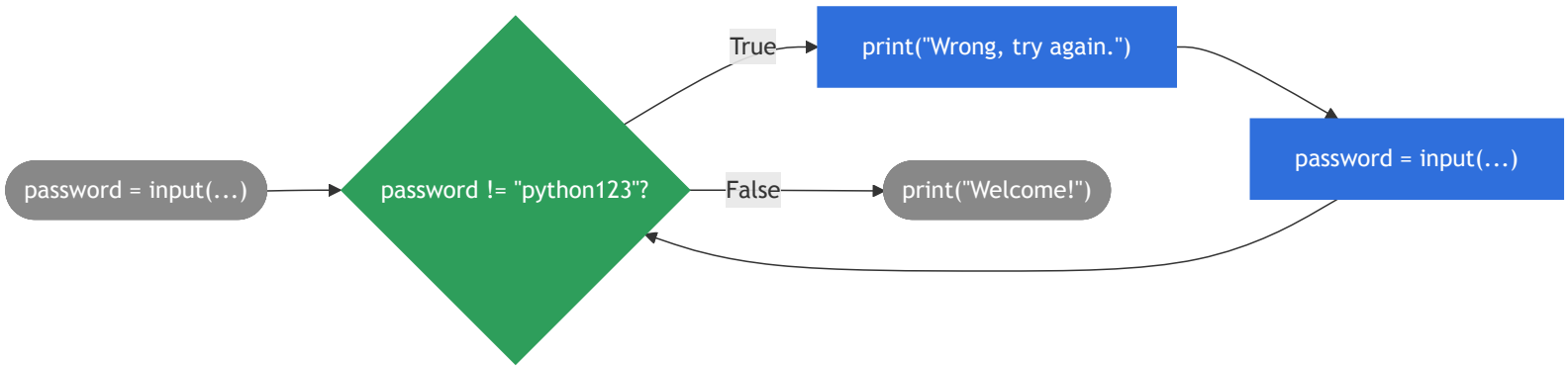
```
2000 is a leap year
2004 is a leap year
2008 is a leap year
2012 is a leap year
2016 is a leap year
2020 is a leap year
2024 is a leap year
2028 is a leap year
2032 is a leap year
2036 is a leap year
2040 is a leap year
Next leap year: 2044
```

04 - Anatomy of a `while` Loop

```
password = input("Password: ")
while password != "python123":
    print("Wrong, try again.")
    password = input("Password: ")
print("Welcome!")
```

- `while` – the keyword that starts the loop
- `password != "python123"` – the **condition**. Python checks it again before every iteration
- `:` + indent – the end of the first line. The indented code is the **body**
- We can't know how many tries it takes. That's when `while` makes sense
- ⚠️ The body **must change** what the condition looks at. Otherwise the loop runs forever

How a `while` Loop Runs



In Plain English

while this condition is true:

    keep doing something

WEEK 5 – LOOPS

Exercise 3 – Savings Goal · 10 min

✍️ You start with **0** euros and save **15** euros every week.

Use a `while` loop to keep saving until you have **at least 100** euros.

- Each week, print the week number and your savings.
- At the end, print how many weeks it took.

Syntax reminder:

```
while condition:
    # runs again and again while condition is True
    # change something, so it ends!
```

Expected output:

```
Week 1: 15 euros
Week 2: 30 euros
Week 3: 45 euros
Week 4: 60 euros
Week 5: 75 euros
Week 6: 90 euros
Week 7: 105 euros
Goal reached in 7 weeks!
```

Exercise 3 – Advanced

🔵 Finished early? Save **10** euros in week 1, then **5 more** each week (10, 15, 20, …) until you have **at least 100** euros.

Print each week's amount and total, then how many weeks it took.

Syntax reminder:

```
while condition:
    total = total + amount
    amount = amount + 5
```

Expected output:

```
Week 1: saved 10, total 10 euros
Week 2: saved 15, total 25 euros
Week 3: saved 20, total 45 euros
Week 4: saved 25, total 70 euros
Week 5: saved 30, total 100 euros
Goal reached in 5 weeks!
```

`for` or `while`?

Pick the one that matches **how** you want to repeat.

`for` – when you know what you're looping over

- a list of items
- a string, letter by letter
- a range of numbers, like `range(5)`
- any collection with a known end

`while` – when you keep going until something changes

- until the user types `"quit"`
- while a condition is still `True`
- as long as there's input to read
- until a goal is reached

`for` is for repeating **a known number of times**. `while` is for repeating **until a condition changes**.

05 - `break` and `continue`

Two keywords that change the flow of a loop.

`break` – exits the loop immediately

Python stops looping the moment it reaches `break`, even if items are left.

Use it when:

- we found what we were looking for
- something went wrong, so we stop
- the user typed `"quit"`

`continue` – skips the rest of this iteration

This iteration ends right there, and Python jumps straight to the next item.

Use it when:

- data is invalid or empty, so we ignore it
- an item doesn't match what we need
- there's a special case we don't care about

`break` vs. `continue` in Code

`break` – stop the loop entirely

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output: `0, 1, 2, 3, 4`

The loop quits the moment `i` reaches `5`.

`continue` – skip this iteration, keep going

```
for i in range(10):
    if i == 5:
        continue
    print(i)
```

Output: `0, 1, 2, 3, 4, 6, 7, 8, 9`

The loop skips `5`, but carries on through `9`.

`break` **leaves** the loop. `continue` **jumps** to the next iteration.

Exercise 4 – Squares Between 10 and 50 · 10 min

✍️ Print the squares of 1, 2, 3, … one per line.

1. Use `for` with `range(1, 100)`
2. Use `continue` to **skip** squares smaller than 10
3. Use `break` to **stop** when a square is bigger than 50

Syntax reminder:

```
for i in range(start, stop):
    if condition:
        continue  # skip to the next i
    if condition:
        break     # leave the loop
```

Expected output:

```
4 * 4 = 16
5 * 5 = 25
6 * 6 = 36
7 * 7 = 49
```

Exercise 4 – Advanced

🔵 Finished early? Do the same with **cubes** (`n * n * n`):

1. Use `for` with `range(1, 100)`
2. Skip cubes smaller than **10**
3. When a cube is bigger than **500**, print `Stopped at` and the number, then stop

Syntax reminder:

```
if condition:
    print("...")  # runs first
    break         # then leaves the loop
```

Expected output:

```
3 * 3 * 3 = 27
4 * 4 * 4 = 64
5 * 5 * 5 = 125
6 * 6 * 6 = 216
7 * 7 * 7 = 343
Stopped at 8
```

Break (15 min) 

06 - Common loop patterns 💡

Lists, strings and nested loops

Sneak Peek: What Is a List?

A **list** holds many values in order, inside square brackets `[ ]`, separated by commas.

- One variable instead of `name1`, `name2`, `name3`
- The values are called **items**
- `len()` tells you how many items there are
- We'll learn much more about lists later

```
names = ["Anna", "Bruno", "Carla"]

print(names)
# ['Anna', 'Bruno', 'Carla']
print(len(names))   # 3
```

Looping Over a List

You can loop over a list with `for`, just like over a `range()`.

- No `range()` needed, just the list itself
- The loop variable takes each item in turn
- The loop runs once per item, so you never need to count

```
names = ["Anna", "Bruno", "Carla"]

for name in names:
    print("Hello,", name)
```

```
Hello, Anna
Hello, Bruno
Hello, Carla
```

It's the same `for ... in ...` pattern as before.

Looping Over a String

A string is a sequence of letters, so you can loop through it the same way.

The code:

```
for letter in "Python":
    print(letter)
```

The output:

```
P
y
t
h
o
n
```

Same `for ... in ...` pattern. Python treats the string as a sequence of its characters.

Pattern 1 – Filtering

Filtering means keeping only the items that match a condition.

- Loop over every item
- Use `if` to check the condition
- Print only the items that pass

```
scores = [12, 45, 7, 30, 18]

for score in scores:
    if score > 15:
        print(score)
```

```
45
30
18
```

WEEK 5 – LOOPS

Exercise 5 – Skip the Zeros · 10 min

✍️ Print each score from the list, but **skip** any zeros.

```
scores = [10, 0, 25, 0, 8, 0, 3]
```

Syntax reminder:

```
for item in my_list:
    if condition:
        print(item)
# != means "is not equal to"
```

Expected output:

```
10
25
8
3
```

Exercise 5 – Advanced

🔵 Finished early? Clean up a messy shopping list:

```
items = [" milk", "", "bread ", "   ", "eggs"]
```

- **Skip** items that are empty or only spaces
- Print the others without spaces, starting with a capital letter

Syntax reminder:

```
text.strip()       # removes spaces at both ends
text.capitalize()  # first letter upper case
# an empty string "" counts as False
```

Expected output:

```
Milk
Bread
Eggs
```

Pattern 2 – Find

Find means saving the first item that matches a condition in a variable.

- **Before** the loop: create the variable
- **Inside**: when an item matches, save it and `break`
- **After**: use the variable

```
scores = [12, 45, 7, 30, 18]

first_big = 0
for score in scores:
    if score > 40:
        first_big = score
        break

print(f"First big score: {first_big}")
```

```
First big score: 45
```

Exercise 6 – Wrong Phone Number · 10 min

✍️ Valid phone numbers have exactly **11 digits**. Print the **first** wrong one **after** the loop.

```
phones = ["01761234567", "01523456789",
          "01709876543", "0162345678",
          "01601234567", "015712345678"]
```

Expected output:

```
First wrong number: 0162345678
```

Syntax reminder:

```
found = ""
for item in my_list:
    if condition:
        found = item
        break
# len(text) gives the number of characters
```

Exercise 6 – Advanced

🔵 Finished early? Use the same list, but also:

- Print the wrong number's **position** in the list: the first number is 1, the second is 2, …
- If every number is fine, print `All numbers look fine`

Syntax reminder:

```
position = 0
for item in my_list:
    position = position + 1
# after the loop: if found == "": ...
```

Expected output:

```
First wrong number: 0162345678 (position 4)
```

Pattern 3 – Accumulating

Accumulating means adding up all the items.

- **Before** the loop: start the total at `0`
- **Inside**: add each item to the total
- **After**: use the total

```
scores = [12, 45, 7, 30, 18]

total = 0
for score in scores:
    total = total + score

print(f"Total: {total}")
```

```
Total: 112
```

WEEK 5 – LOOPS

Exercise 7 – Average Score · 10 min

✍️ Use a loop to add up the scores. Then divide the total by the number of scores and print the **average**.

```
scores = [8, 6, 10, 4, 7]
```

Syntax reminder:

```
total = 0
for item in my_list:
    total = total + item
# len(my_list) gives the number of items
```

Expected output:

```
Average: 7.0
```

WEEK 5 – LOOPS

Exercise 7 – Advanced

🔵 Finished early? Only scores of **5 or more** pass. Print how many scores passed and their **average**.

```
scores = [8, 2, 10, 4, 6]
```

Syntax reminder:

```
for item in my_list:
    if condition:
        total = total + item
        count = count + 1  # len() would count all
```

Expected output:

```
Passing scores: 3
Average of passing scores: 8.0
```

Pattern 4 – Finding the Max

Finding the max means keeping the biggest item seen so far.

- **Before** the loop: start `highest` at `0`
- **Inside**: if an item is bigger, it becomes the new `highest`
- **After**: `highest` holds the biggest item

```
scores = [12, 45, 7, 30, 18]

highest = 0
for score in scores:
    if score > highest:
        highest = score

print(f"Highest: {highest}")
```

```
Highest: 45
```

Patterns 2–4 have the same shape: **create** a variable before the loop, **update** it inside, **use** it after.

Nested Loops

A loop inside a loop – each outer pass runs the inner loop **completely**. Here, a list holds other lists, like the rows of a table.

The code:

```
table = [[1, 2, 3],
         [4, 5, 6]]

for row in table:
    for cell in row:
        print(cell)
```

The output:

```
1
2
3
4
5
6
```

The inner loop runs once for every item of the outer loop – 2 rows × 3 cells = 6 lines.

Step by Step: Nested Loops

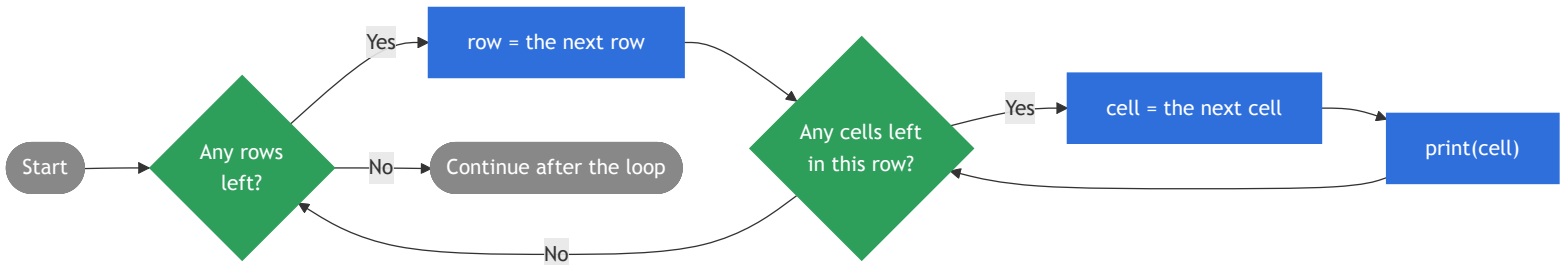
```
for row in table:
    for cell in row:
        print(cell)
```

`row` stays the same while the inner loop goes through **every** cell in it.

Then the outer loop moves to the next row, and the inner loop starts again.

Iteration	`row` is	`cell` is	It prints
1st	`[1, 2, 3]`	`1`	`1`
2nd	`[1, 2, 3]`	`2`	`2`
3rd	`[1, 2, 3]`	`3`	`3`
4th	`[4, 5, 6]`	`4`	`4`
5th	`[4, 5, 6]`	`5`	`5`
6th	`[4, 5, 6]`	`6`	`6`

How Nested Loops Run



When the inner loop runs out of cells, it goes back to the outer loop for the next row.

WEEK 5 – LOOPS

What We Learned Today

✅ `for` repeats a known number of times, `while` repeats until a condition changes

✅ `range()` counts for you, and always stops **before** the stop number

✅ Indented lines run **every pass**; unindented lines run once, **after** the loop

✅ `break` leaves the loop, `continue` skips the current iteration

✅ A **list** `[ ]` holds many values. `for` can loop over lists and strings

✅ Common patterns: **filter**, **find**, **accumulate**, and **find the max**

✅ **Nested loops** run a whole inner loop for each outer pass, like rows and cells of a table

Just like with `if` / `elif` / `else`, the right loop depends on the task. Practice is how you'll learn which one fits each job, and why. Now it's time to practise! 💪

QUESTIONS?

07 - Homework! 

📌 Commit your work to GitHub when you're done!

WEEK 5 – LOOPS

Homework 1 – Count to 20

🏠 Print the numbers from **1 to 20**, one per line.

Expected output:

```
1
2
3
...
19
20
```

WEEK 5 – LOOPS

Homework 2 – Say Hi to Your Friends

🏠 For each friend in the list, print `Hi, <name>`:

```
friends = ["Anna", "Bruno", "Carla"]
```

Expected output:

```
Hi, Anna
Hi, Bruno
Hi, Carla
```

Homework 3 – Countdown by Twos

🏠 Count down from **20 to 2** in steps of **2**, then print `"Done!"`.

Expected output:

```
20
18
16
14
12
10
8
6
4
2
Done!
```

WEEK 5 – LOOPS

Homework 4 – Add It Up

🏠 Add up all the numbers from **1 to 100** and print the total.

Expected output:

```
5050
```

Homework 5 – Count the Vowels

🏠 How many vowels (`a` , `e` , `i` , `o` , `u`) are in the word `"programming"` ?

Print the count.

Expected output:

```
3
```

Homework 6 – Even or Odd?

🏠 For each number in the list, print whether it's even or odd:

```
numbers = [3, 8, 15, 22, 7]
```

New: `%` (called **modulo**) gives the remainder left over after dividing:

- `10 % 3` → `1` (10 ÷ 3 = 3, with 1 left over)
- `8 % 2` → `0` (8 divides by 2 exactly)
- `7 % 2` → `1` (odd numbers leave 1 over)

So `n % 2 == 0` means `n` divides evenly by 2: it's even.

Expected output:

```
3 is odd
8 is even
15 is odd
22 is even
7 is odd
```

WEEK 5 – LOOPS

Homework 7 – Number Grid

🏠 Print a 5 × 5 grid where each row shows `1 2 3 4 5`.

Hint: by default, every `print()` ends with a new line. That's why each call starts a fresh line. `end="..."` changes that:

- `print(n, end=" ")` → the value and a space, on the same line
- `print()` on its own → just a new line, to end the row

Expected output:

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

WEEK 5 – LOOPS

Homework 8 – Multiplication Table

🏠 Print a multiplication table for **1 to 5**.

An empty line after each group makes it easier to read.

Expected output (first rows):

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5

2 * 1 = 2
...
```

WEEK 5 – LOOPS

Homework 9 – The Largest Number

🏠 Find and print the largest number in the list:

```
numbers = [4, 19, 2, 8, 11]
```

Expected output:

```
19
```

WEEK 5 – LOOPS

Homework 10 – Numbers and Squares

🏠 For each number from **1 to 10**, print the number and its square, separated by a space.

Expected output:

```
1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
10 100
```

WEEK 5 – LOOPS

Homework 11 – FizzBuzz

🏠 For each number from **1 to 20**:

- Multiple of 3 → print `Fizz`
- Multiple of 5 → print `Buzz`
- Multiple of both → print `FizzBuzz`
- Otherwise → print the number

Expected output (first lines):

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
...
```



See you next week!